prevention.py M

backend > app > rules > prevention.py

```
there's no Fraud & Bad-Faith Detection Agent backstop (Policy doc Part 2, Ru

# Categories that have a hard evidence requirement at filing time (Rule 4). I
# and No-Show are captured automatically by the Evidence Collection Agent, s
# rarely fires for them in practice — Property Damage is the real case it ex
CATEGORIES_REQUIRING_EVIDENCE_AT_FILING = {DisputeCategory.PROPERTY_DAMAGE}

# ---------------------------------------------------------------------------
# Rule 3 safety keyword backstop
# ---------------------------------------------------------------------------
# Triage's `is_safety` flag is the primary signal, but LLMs can miss or misr
# safety-critical language. This plain keyword list is a deterministic backs
# *either* Triage or this scan fires, the claim is treated as a safety repor
# bypassed to a human.
#
# Matching strategy: `\b` (word boundary) on the *left* so that "assault" do
# match inside a longer word like "assaulted" — wait, we *want* to catch
# inflected forms. So we anchor only the left edge and let the keyword match
# a prefix: "assault" → "assaulted", "harass" → "harassment", "threat" →
# "threatened", "drunk" → "drunken". The left `\b` still prevents matching
# "truncated" (starts with "trun") or "threatened" from "three".
#
# "995" is special: it must match only as a standalone number, not inside a
# longer number like "99521". So it gets `\b` on both sides.
SAFETY_KEYWORDS: tuple[str, ...] = (
    "assault",
    "threat",
    "weapon",
    "knife",
    "drunk",
    "harass",
    "followed me",
    "police",
    "995",
)

# Build the regex: left \b for all keywords; "995" also gets a right \b.
_pattern_parts = []
for _kw in SAFETY_KEYWORDS:
    if _kw == "995":
        _pattern_parts.append(rf"\b{re.escape(_kw)}\b")
    else:
        _pattern_parts.append(rf"\b{re.escape(_kw)}")
_SAFETY_                                                    ASE)
```

↑ 1 / 7 ↓ ← File 3 / 3 → Keep Undo

CodeBuddy

Safety-word backstop (C...

I should use `\b` only at the start, and let the keyword match any continuation, OR use a different approach. Let me rethink: the form says keywords like "assault, threat, weapon, knife, drunk, harass, followed me, police, 995". The intent is to catch these safety terms in any grammatical form. So I should match the stem, not require exact word boundary at the end.

Let me fix the regex to use `\b` at the start but allow trailing letters (so "harass" matches "harassment", "assault" matches "assaulted", "threat" matches "threatened"):

Read prevention.py L30-59

I'll replace the regex to use `\b` only at the start (prefix match) so inflected forms are caught, but also make sure near-misses like "truncated" still don't fire. Let me think through each near-miss with a prefix-only approach:

- "truncated" — does it start with `\bdrunk`? No, "truncated" starts with "trun". Good, no match.
- "harsh" — `\bharass`? "harsh" doesn't contain "harass". Good.
- "policy" — `\bpolice`? No, "policy" doesn't start with "police". Good.
- "99521" — `\b995`? "99521" starts with "995" at a word boundary (start of digits). This WOULD match! Bad.
- "followed the GPS" — `\bfollowed me`? No. Good.

So the only problem case is "99521" which would match with prefix-only. I need `\b` at the end for "995" but prefix for the others. Better approach: use `\b` at start for all, and `\b` at end only for "995":

prevention.py (Modify) +19 -9 View Diff

Waiting for the model response...

Task List 3/4 File List (3)

Task 3 is being executed

Ask or type "/" for commands

Craft Auto Skills

AI-generated content, for reference only

ryde-dispute-resolution-tencent

Open Agents

test_data_generation.py U

backend > tests > test_data_generation.py

```python
"""Unit tests for the data-generation pipeline's pure functions.

Covers:
  • geo.py — haversine distance, route length, GPS interpolation, route pert
    detour-waypoint geometry, neighbourhood sampling.
  • fares.py — Ryde's real fee figures ($1.25 / $1.47 platform fee, $4.50 no
    $40/$120/$200 damage tiers) and fare assembly logic.
  • golden_cases.py — the deterministic oracle that labels each golden case.

No database or network calls — every function tested here is pure Python.
"""

import random
from datetime import UTC, datetime, timedelta
from math import isclose

import pytest

from data_generation.pipeline.fares import (
    BASE_FARE_MIN,
    BASE_FARE_PER_KM,
    DESTINATION_CHANGE_FEE,
    NO_SHOW_FEE,
    PLATFORM_FEE_HIGH,
    PLATFORM_FEE_LOW,
    PLATFORM_FEE_THRESHOLD,
    PROPERTY_DAMAGE_FEES,
    TAX_RATE,
    apply_no_show_fee,
    apply_property_damage_fee,
    generate_fare,
)
from data_generation.pipeline.geo import (
    SINGAPORE_NEIGHBORHOODS,
    _EARTH_RADIUS_KM,
    detour_waypoint,
    haversine_km,
    interpolate_gps_trace,
    perturb_route,
    route_length_km,
    sample_pickup_dropoff,
)
from dat
    _mir
    _no_show_outcome,
```

Keep ⇧⌥Y Undo ⇧⌥N

1 / 2 File 4 / 4 Keep Undo

CodeBuddy

Safety-word backstop (Craft... New Chat

Task List 4/4 done

- ~~Write test_data_generation.py: geo.py distance/perturbation tests~~
- ~~Write test_data_generation.py: fares.py fee figures tests~~
- ~~Write test_data_generation.py: golden_cases.py labelling edge tests~~
- ~~Run tests, screenshot green run~~

All 67 tests green. Here's the summary.

`backend/tests/test_data_generation.py` — 67 tests, 0 failures, no DB/network

geo.py — distance maths and route perturbation (23 tests):

Area	Tests	Key cases
`haversine_km`	4	zero distance, known Singapore distance (Orchard→Raffles Place ~3.2 km), symmetry, antipodal = π·R
`route_length_km`	3	single segment equals haversine, multi-segment sum, degenerate single-point = 0
`interpolate_gps_trace`	4	first/last points match endpoints, timestamps span start→end, min 2 samples f short durations, sample count

Task List 4/4 File List (4)

All tasks completed

Ask or type "/" for commands

Craft Auto Skills

AI-generated content, for reference only

Ln 1, Col 1 Spaces: 4 UTF-8 LF Python WeCom

ryde-dispute-resolution-tencent

Open Agents

EXPLORER

RYDE-DISPUTE-RESOLUTION-TENCENT

backend

.ruff_cache
.venv
app
data_generation
migrations
scripts
tests
__pycache__
__init__.py
conftest.py
test_adjudication.py
test_advocate_tools.py
test_advocates.py
test_agent_retry.py
test_cases_api.py
test_data_generation.py U
test_demo_api.py
test_escalation_compliance.py M
test_evidence_api.py
test_evidence_collection.py
test_fixtures.py
test_golden_run.py
test_image_analysis.py
test_judge.py
test_l2_review.py
test_onemap.py
test_party_outcome.py
test_pipeline.py
test_prevention.py M
test_state_machine.py

OUTLINE

TIMELINE



Hi Buddy

Waiting for work...

Show All Commands ⇧ ⌘ P

Go to File ⌘ P

Start Debugging F5

Find in Files ⇧ ⌘ F

Open Settings ⌘ ,

CodeBuddy

o explain and critique backe...

Here is my analysis and the fix.

Critique of `escalation.py` and `compliance.py`

Q1: Can a decision ever skip the escalation gate?

No. The escalation gate is mandatory and unreachable by bypass.

Evidence from the call chain in `_run_adjudication` (`adjudication.py:171–175`):

- `run_escalation_gate` is called unconditionally on every Judge verdict — there is no branch or early return that skips it.
- The only paths to `CaseState.DECIDED` or `CaseState.ESCALATED` from `CaseState.ADJUDICATING` go through `_run_adjudication` (confirmed by the state machine at `state_machine.py:38`).
- The only other path to `DECIDED` is from `ESCALATED` or `SAFETY_BYPASSED` via L2 review (`l2_review.py`), which intentionally bypasses the gate — a human reviewer doesn't need a confidence threshold.

The gate's three escalation triggers (`bucket_escalate`, `confidence_escalate`, `judge_requested_escalation`) are all OR'd in `routes_to_l2`, and any one of them routes to L2. The Judge's `escalate_to_l2` request can *add* an escalation but never remove one (it only sets `judge_requested_escalation=True`). This is correctly designed — the model cannot talk its way out of escalation.

Q2: Can the compliance check ever author a decision?

No. The compliance check is strictly a veto — it returns `str | None`, never a verdict or remedy.

Evidence:

- `compliance_check` (`compliance.py:121–124`) joins violations into a string or returns `None`. It never constructs or modifies a `JudgeVerdict` or `DecisionObject`.

2 file(s) Checkpoint 1 Keep Undo View Diff

Ask or type "/" for commands

Craft Auto Skills

AI-generated content, for reference only

dev-b/party-app-polish* 0 0

WeCom

Write README and demo script

→ deployment diagram → close. Includes a pre-flight checklist, a "what not to show" list (no real users, no cost-savings claims, no Ryde marks), and answers to four anticipated judge questions.

Both files use the policy-vs-design-honest framing the hackathon cares about: the $4.50 no-show fee, $40/$120/$200 cleaning tiers, and 30-day window are labelled real-Ryde-sourced; the 10%/25% route-deviation tiers, 100 m arrival radius, and 2 km traffic-match radius are labelled as our own design choices.

Files written:

- `copy.md` (110 lines)
- `demo-script.md` (56 lines)

Note: per `RydeResolve-Execution-Plan.md` §0, the next outstanding steps on the same list (#4 owner) are the cover image (`submission/cover.png`, row #5) and the actual recording (Dev A, row #8) — both already set up to use the script you now have.

Process

Done. The two submission files are saved, viewable in the panel above, and ready to drop straight into the form and the video recording.

copy.md
7.1 KB

demo-script.md
9.3 KB

View all artifacts (2) ›

Total used ◇ 4.3 Auto

What can I help you with today? @ add context, / enable skills and commands

Default Permissions

Auto

Friends In, Credits ...
Invite a friend and earn 100 cr...

srivastavashiv0112@gm...

Content generated by AI. Please verify important information.

copy.md | demo-script.md

demo-script.md

RydeResolve — 5–8 minute demo script

> Target length: **~7 minutes**. Built for the video recording step in the Execution Plan (Dev A, after the live deploys are verified). Shot against the live URLs; no waiting on a model while recording.
>
> Cast: **one narrator** (also the on-camera presenter; can be the same person as Dev B). No second voice required. The UI does most of the talking.
>
> Pre-flight (5 min, do this first):
>
> 1. `cd backend && uv run python -m scripts.shift_dates` so trips fall inside the 30-day claim window.
> 2. `curl https://<backend>/healthz` — get the cold cache out of the way.
> 3. Open the frontend in two tabs: one **light** theme for the persona flow, one **dark** theme for the System Trace and Accuracy screens.
> 4. Open the **System trace** tab on `CASE-0153` (the route deviation used in the trace screenshot) and refresh so the trace is cached.
> 5. Have the **Accuracy** page open on a 3-minute-old golden run. Don't re-run it on camera — that's slow and the number on screen should match the README (22/22).
> 6. Quiet room. No notifications. 1080p or higher, 16:9, ≥ 24 fps.

Cue sheet (≈ 7:00 total)

Time	Segment	On screen	What you say (approximate)
0:00 – 0:25	**Cold open**	Title card — "RydeResolve — a fair hearing for	"What does an *actual* hearing look like for a ride-hailing dispute? Not a chatbot that picks a side, not a single model that reads a